



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Compiladores

Eliminación de Recursividad Izquierda,
Notación EBNF y Conjunto First

Tarea 3

Presenta:

Ramírez Juárez María Fernanda - 321204747

Rojo Peña Manuel Ianluck - 118005762

Profesor:

Pablo Martínez Yessica Janeth

Ayudante:

Rojas Fuentes Andrea

Grupo: 7010, 2027-1

Fecha de entrega:

21 de septiembre, 2026

1. Para cada una de las siguientes gramáticas:

- Elimina la recursión por la izquierda.
- Escriba la gramática en notación EBNF.
- Dibuje los diagramas de sintaxis.
- Obtenga el conjunto First.

- Gramática 1:

$$\begin{aligned}lexp &\rightarrow atom \mid list \\atom &\rightarrow num \mid id \\list &\rightarrow (lexp-seq) \\lexp-seq &\rightarrow lexplexp-seq \mid lexp\end{aligned}$$

Eliminación por izquierda:

En esta gramática se observa que ninguna de sus reglas presenta recursividad por la izquierda, las producciones directas como:

$$\begin{aligned}lexp &\rightarrow atom \mid list \\atom &\rightarrow num \mid id\end{aligned}$$

son derivaciones simples, La única regla recursiva es la de secuencias:

$$lexp-seq \rightarrow lexp, lexp-seq \mid lexp$$

Dado que el no terminal *lexp-seq* se llama a sí mismo en el extremo derecho de la producción, se trata de una recursividad por la derecha. Por lo tanto, la gramática ya cumple con la condición solicitada y no requiere aplicar eliminación por izquierda.

Notación EBNF:

La producción

$$lexp-seq \rightarrow lexp \ lexp-seq \mid lexp$$

genera simplemente una secuencia de una o más apariciones de *lexp* concatenadas directamente (sin coma ni paréntesis entre ellas). En **EBNF**, la expresamos usando la cerradura de repetición.

Las demás producciones se mantienen estructuralmente idénticas, ya que solo utilizan la alternancia directa.

Por lo tanto, la Gramática 1 en **EBNF** queda así:

$$\begin{aligned}lexp &\rightarrow atom \mid list \\atom &\rightarrow num \mid id \\list &\rightarrow (\ lexp-seq \) \\lexp-seq &\rightarrow lexp \{ \ lexp \ }\end{aligned}$$

Diagrama de sintaxis:

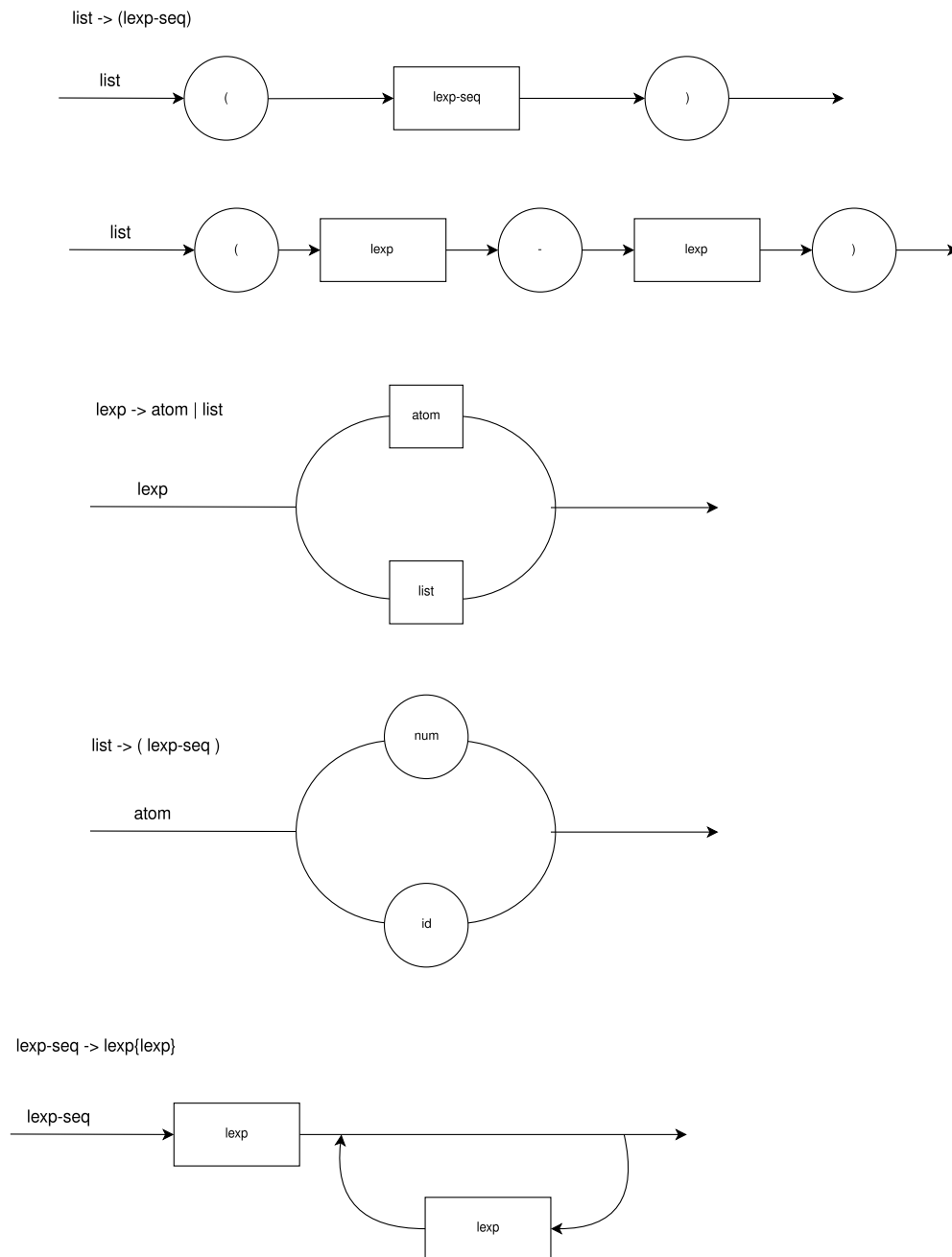


Figure 1: Diagrama de sintaxis para la Gramática 1

Conjunto FIRST:

Paso 1. Terminales

$$\text{FIRST}(\textit{atom}) = \{\text{num}, \text{id}\}$$

Paso 2. *list*

Su producción empieza con el terminal (por lo que no hay nada más que hacer, ya que el paréntesis no es anulable, el análisis se detiene en el primer símbolo, el resto del lado derecho es irrelevante.

$$\text{FIRST}(\textit{list}) = \{($$

Paso 3. *lexp*

Aquí sí hay dos alternativas y se unen los FIRST de cada una:

$$\begin{aligned} \text{FIRST}(\textit{lexp}) &= \text{FIRST}(\textit{atom}) \cup \text{FIRST}(\textit{list}) \\ &= \{\text{num}, \text{id}\} \cup \{(\\ &= \{\text{num}, \text{id}, (\end{aligned}$$

Paso 4. *lexp-seq*

La producción en EBNF es

$$\textit{lexp-seq} \rightarrow \textit{lexp} \{ \textit{lexp} \}$$

Aquí ya no hay alternativas (|) que considerar: es una única secuencia formada por un *lexp* obligatorio seguido de una repetición opcional de más *lexp*. El símbolo con el que *puede empezar* esta producción está determinado únicamente por el primer elemento obligatorio, es decir *lexp*; la parte $\{ \textit{lexp} \}$ es opcional y por lo tanto no aporta símbolos iniciales adicionales (aunque casualmente en este caso serían los mismos, por ser el mismo no terminal repetido).

$$\text{FIRST}(\textit{lexp-seq}) = \text{FIRST}(\textit{lexp}) = \{\text{num}, \text{id}, ($$

Respuesta final

$$\begin{aligned} \text{FIRST}(\textit{lexp}) &= \{\text{num}, \text{id}, (\\ \text{FIRST}(\textit{atom}) &= \{\text{num}, \text{id}\} \\ \text{FIRST}(\textit{list}) &= \{(\\ \text{FIRST}(\textit{lexp-seq}) &= \{\text{num}, \text{id}, (\end{aligned}$$

- Gramática 2:

$$\begin{aligned} S &\rightarrow A B \\ A &\rightarrow \text{begin } S \text{ end } B \text{ theend} \mid \varepsilon \\ B &\rightarrow \text{var } L \text{:tipo} \mid B \text{fvar} \mid \varepsilon \\ L &\rightarrow L, \text{id} \mid \text{id} \end{aligned}$$

Eliminación por izquierda:

- $S / A: A \rightarrow \varepsilon$ permite que S derive directamente en B , ninguna producción de B empieza en S ni en A , así que no se propaga ninguna recursión hacia S ni hacia A .
- B , por $B \text{ to } B \text{ fvar}$ tenemos:

$$\begin{aligned}\alpha_1 &= \text{fvar}, \\ \beta_1 &= \text{var } L : \text{tipo}, \\ \beta_2 &= \varepsilon\end{aligned}$$

Cada β se acompaña del nuevo no-terminal B' , y $\beta_2 = \varepsilon$ produce simplemente $B \rightarrow B'$:

$$\begin{aligned}B &\rightarrow \text{var } L : \text{tipo } B' \mid B' \\ B' &\rightarrow \text{fvar } B' \mid \varepsilon\end{aligned}$$

Ese $B \rightarrow B'$ es una producción unitaria y la podemos dejar así para hacer la gramática más limpia:

$$B \rightarrow \text{var } L : \text{tipo } B' \mid \text{fvar } B' \mid \varepsilon$$

- L , por $L \rightarrow L , \text{id}$ tenemos:

$$\begin{aligned}\alpha &= , \text{id} \\ \beta &= \text{id}\end{aligned}$$

Entonces:

$$\begin{aligned}L &\rightarrow \text{id } L' \\ L' &\rightarrow , \text{id } L' \mid \varepsilon\end{aligned}$$

Gramática Resultante:

$$\begin{aligned}S &\rightarrow A B \\ A &\rightarrow \text{begin } S \text{ end } B \text{ theend} \mid \varepsilon \\ B &\rightarrow \text{var } L : \text{tipo } B' \mid B' \\ B' &\rightarrow \text{fvar } B' \mid \varepsilon \\ L &\rightarrow \text{id } L' \\ L' &\rightarrow , \text{id } L' \mid \varepsilon\end{aligned}$$

Notación EBNF:

En vez de partir de las reglas ya transformadas (B' y L'), conviene derivar **EBNF** a partir del lenguaje que genera cada regla original, igual que se hizo con *lexp-seq* en la Gramática 1.

Usamos $[]$ para lo opcional y $\{ \}$ para lo repetible:

- $L \rightarrow L , \text{id} \mid \text{id}$ genera una secuencia de uno o más **id** separados por comas, por lo que su cerradura de repetición es:

$$L \rightarrow \text{id} \{ , \text{id} \}$$

- $B \rightarrow \text{var } L : \text{tipo} \mid B \text{ fvar} \mid \varepsilon$ genera nada con cero o más **fvar**, o bien, **var** $L : \text{tipo}$ seguido de cero o más **fvar**, es decir que la primera es opcional mientras que **fvar** se repite:

$$B \rightarrow [\text{var } L : \text{tipo}] \{ \text{fvar} \}$$

- $A \rightarrow \text{begin } S \text{ end } B \text{ theend} \mid \varepsilon$ toda producción es opcional:

$$A \rightarrow [\text{begin } S \text{ end } B \text{ theend}]$$

- $s \rightarrow A B$ no cambia, es un simple concatenación.

Gramática Resultante:

$$S \rightarrow A B$$

$$A \rightarrow [\text{begin } S \text{ end } B \text{ theend}]$$

$$B \rightarrow [\text{var } L : \text{tipo}] \{ \text{fvar} \}$$

$$L \rightarrow \text{id} \{ , \text{id} \}$$

Diagrama de sintaxis:

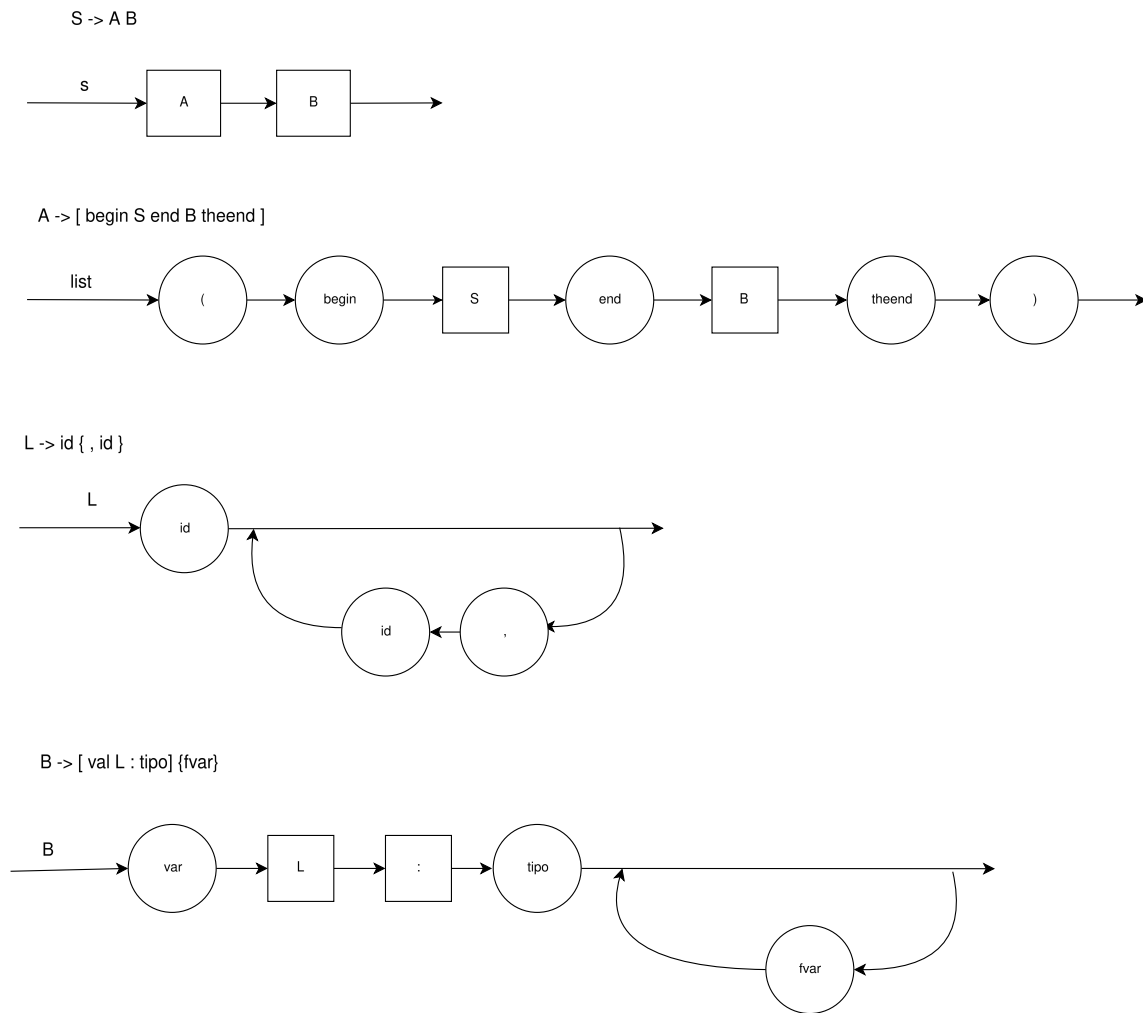


Figure 2: Diagrama de sintaxis para la Gramática 2

Conjunto FIRST:

Paso 1. L

La producción en EBNF es

$$L \rightarrow \text{id} \{ , \text{id} \}$$

El símbolo obligatorio al inicio es **id**, la parte $\{ , \text{id} \}$ es opcional y no aporta nada nuevo al FIRST. Como **id** es un terminal, L no es anulable, siempre debe generar al menos ese **id**, entonces:

$$\text{FIRST}(L) = \{\text{id}\}$$

Paso 2. B

La producción es

$$B \rightarrow [\text{var } L : \text{tipo}] \{ \text{fvar} \}$$

Aquí hay dos partes opcionales, así que hay que unir los símbolos iniciales de cada una:

- Si el bloque $[\text{var } L : \text{tipo}]$ está presente, empieza con el terminal **var**.
- Si el bloque está ausente, lo que sigue es $\{ \text{fvar} \}$, que empieza con **fvar** si hay al menos una repetición, o puede no aportar nada si la repetición es cero.

Como ambos bloques opcionales pueden estar ausentes simultáneamente, B sí es anulable ($B \Rightarrow^* \varepsilon$).

$$\text{FIRST}(B) = \{\text{var}\} \cup \{\text{fvar}\} = \{\text{var}, \text{fvar}\}$$

Paso 3. A

La producción es

$$A \rightarrow [\text{begin } S \text{ end } B \text{ theend}]$$

Toda la producción es un único bloque opcional que, de estar presente, empieza con el terminal **begin**, no importa que dentro aparezca S (que a su vez depende de A), el primer símbolo. Si el bloque existe, siempre es el terminal **begin**, así que no hay ninguna dependencia circular real que resolver aquí. Como el bloque completo es opcional, A es **anulable** directamente.

$$\text{FIRST}(A) = \{\text{begin}\}$$

Paso 4. S

La producción es

$$S \rightarrow A B$$

Es una concatenación simple, el FIRST de S empieza con el FIRST de A , pero como A es anulable, también hay que agregar el FIRST de B , el símbolo que le sigue:

$$\begin{aligned} \text{FIRST}(S) &= \text{FIRST}(A) \cup \text{FIRST}(B) \quad (\text{porque } A \text{ es anulable}) \\ &= \{\text{begin}\} \cup \{\text{var}, \text{fvar}\} \\ &= \{\text{begin}, \text{var}, \text{fvar}\} \end{aligned}$$

Respuesta final

$$\begin{aligned} \text{FIRST}(S) &= \{\text{begin}, \text{var}, \text{fvar}\} \\ \text{FIRST}(A) &= \{\text{begin}\} \\ \text{FIRST}(B) &= \{\text{var}, \text{fvar}\} \\ \text{FIRST}(L) &= \{\text{id}\} \end{aligned}$$

2. Construya el conjunto First de las siguientes gramáticas:

a.

$$\text{declaracion} \rightarrow \text{tipo } \text{var-list}$$

$$\text{tipo} \rightarrow \text{int} \mid \text{float}$$

$$\text{var-list} \rightarrow \text{id} \mid \text{id} \text{ , } \text{var-list}$$

Paso 1. Terminales

tipo y *var-list* tienen producciones que arrancan directamente con terminales, así que se resuelven sin depender de nadie.

$$\text{FIRST}(\text{tipo}) = \text{FIRST}(\text{int}) \cup \text{FIRST}(\text{float}) = \{\text{int}, \text{float}\}$$

$$\text{FIRST}(\text{var-list}) = \text{FIRST}(\text{id}) \cup \text{FIRST}(\text{id}) = \{\text{id}\}$$

Las dos alternativas de *var-list* empiezan con *id*, así que la unión colapsa a un solo elemento.

Paso 2. *declaracion*

Su única producción es $\text{declaracion} \rightarrow \text{tipo } \text{var-list}$, el primer símbolo es *tipo*, y $\varepsilon \notin \text{FIRST}(\text{tipo})$ porque *tipo* siempre produce *int* o *float*:

$$\text{FIRST}(\text{declaracion}) = \text{FIRST}(\text{tipo}) = \{\text{int}, \text{float}\}$$

Verificación por derivaciones.

Toda cadena del lenguaje se ve así:

$$\text{declaracion} \Rightarrow \text{tipo } \text{var-list} \Rightarrow \text{int } \text{var-list} \Rightarrow \text{int id}$$

$$\text{declaracion} \Rightarrow \text{tipo } \text{var-list} \Rightarrow \text{float } \text{var-list} \Rightarrow \text{float id , id}$$

El primer token es *int* o *float*, nunca *id*, pues este aparece en la *segunda* posición, y FIRST no se entera de lo que pasa en la segunda posición.

Respuesta final

$$\text{FIRST}(\text{declaracion}) = \{\text{int}, \text{float}\}$$

$$\text{FIRST}(\text{tipo}) = \{\text{int}, \text{float}\}$$

$$\text{FIRST}(\text{var-list}) = \{\text{id}\}$$

b.

$$\begin{aligned}
lexp &\rightarrow atom \mid list \\
atom &\rightarrow \text{num} \mid \text{id} \\
list &\rightarrow (\text{lexp-seq}) \\
lexp-seq &\rightarrow \text{lexp} , \text{lexp-seq} \mid \text{lexp}
\end{aligned}$$

Paso 1. Terminales

$$\text{FIRST}(atom) = \{\text{num}, \text{id}\}$$

Paso 2. list

Su producción empieza con el terminal (, no hay nada más que hacer, el resto del es irrelevante.

$$\text{FIRST}(list) = \{(\}$$

Paso 3. lexp

Aquí sí hay dos alternativas y se unen los FIRST de cada una:

$$\begin{aligned}
\text{FIRST}(lexp) &= \text{FIRST}(atom) \cup \text{FIRST}(list) \\
&= \{\text{num}, \text{id}\} \cup \{(\} \\
&= \{\text{num}, \text{id}, (\}
\end{aligned}$$

Paso 4. lexp-seq

A diferencia del caso anterior, ahora ambas alternativas de

$$lexp-seq \rightarrow \text{lexp} , \text{lexp-seq} \mid \text{lexp}$$

comienzan con el mismo no terminal *lexp*, no hay ninguna alternativa que empiece directamente con un terminal, por lo tanto:

$$\text{FIRST}(lexp-seq) = \text{FIRST}(lexp)$$

Como *lexp* no es anulable (no deriva ε), no hace falta considerar lo que sigue después de *lexp* en la primera alternativa; el cálculo se detiene ahí.

$$\text{FIRST}(lexp-seq) = \{\text{num}, \text{id}, (\}$$

Respuesta final

$$\begin{aligned}
\text{FIRST}(lexp) &= \{\text{num}, \text{id}, (\} \\
\text{FIRST}(atom) &= \{\text{num}, \text{id}\} \\
\text{FIRST}(list) &= \{(\} \\
\text{FIRST}(lexp-seq) &= \{\text{num}, \text{id}, (\}
\end{aligned}$$